

When WebSockets Lead to RCE in CurseForge

When WebSockets Lead to RCE in CurseForge - Elliott, Elliott.diy.

- Published: 2025-12-23
- Original: <<https://elliott.diy/blog/curseforge/>>
- Preserved from: <https://elliott.diy/blog/curseforge/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Dec 23, 2025

When WebSockets Lead to RCE in CurseForge



On this page

- Introduction
- Finding the WebSocket

- No Origin Checks
- Proof of Concept
- Step 1: Creating a modpack
- Step 2: Launching with attacker-controlled JVM arguments
- Port Discovery
- Disclosure Timeline
- Final Notes

Introduction

After finding the [SuperShy RCE](#) a few months earlier, I ended up going down a bit of a WebSocket rabbit hole. Out of curiosity, I started poking around to find other applications on my system that regularly expose local WebSocket servers bound to the loopback interface.

I found a handful of interesting apps that will turn into future write-ups pending coordinated disclosure, but the one that stood out immediately was the modding platform CurseForge. CurseForge is one of the most widely used video game modding platforms, with millions of users relying on its desktop launcher to manage and launch modpacks.

Finding the WebSocket

While monitoring local WebSocket traffic, I noticed that every time a modpack was launched, the CurseForge launcher would send the following message over a local WebSocket connection:

```
{
  "args": [
    {
      "MinecraftInstanceGuid": "9eelc6b8-f0f3-441c-b6be-6b03a7a6019a",
      "ResolutionWidth": 1024,
      "ResolutionHeight": 768,
      "LauncherVisibility": "Close",
      "LauncherType": "Classic",
      "AdditionalJavaArguments": ""
    }
  ],
  "type": "method",
  "name": "minecraftTaskLaunchInstance"
}
```

At first glance, this looked like a typical launch message. The detail that stood out was that the WebSocket endpoint was bound to localhost, and the message contained no obvious

authentication or authorization mechanism, such as API keys, session tokens, or per-request validation.

Although the service was bound to localhost, it could be reachable by any website loaded in the user's browser, since browsers allow cross-origin WebSocket connections unless the server explicitly enforces origin checks.

I wrote a small script that re-sent the same WebSocket message, this time with a bogus Origin header set to `elliott.diy`. To my surprise, the launcher accepted the connection, and the modpack launched successfully.

If a random script with a bogus origin header could send WebSocket messages to the CurseForge launcher without restriction, then any website visited by a user could control the launcher as well. From there, I started digging into what was actually hosting this WebSocket server and what else I could do.

No Origin Checks

The server is implemented in `CurseAgent.exe` and starts when the launcher opens. I loaded it into JetBrains decompiler, dotPeek, and quickly confirmed that no origin validation was being attempted at all on the incoming WebSocket connections.

I also started digging into the exposed methods and found several callable actions beyond simply launching a modpack:

- **minecraftGetDefaultLocation** - Returns the default Minecraft installation path
- **createModpack** - Creates a new modpack and returns its GUID
- **minecraftTaskLaunchInstance** - Launches a modpack with a supplied GUID, launcher type, and arbitrary Java arguments

There are many other methods for other games and launcher features, but the above are the most relevant for this write-up.

The biggest concern here is `minecraftTaskLaunchInstance`, as it allows attacker-controlled JVM arguments to be supplied via the `AdditionalJavaArguments` field when launching the game.

Proof of Concept

Live PoC: <https://research.elliott.diy/97bef577>

Source code: <https://github.com/elliott-diy/curseforge>

To see how far this could go, I wrote a simple proof-of-concept. CurseForge supports several other games, and I'm fairly confident similar techniques would work there as well, but I'm a die-hard Minecraft fan, so I stuck with the Minecraft-specific methods. The PoC chains together two exposed WebSocket methods: one to create a new modpack, and another to launch it with attacker-controlled JVM arguments.

Step 1: Creating a modpack

The first step is calling `createModpack`, which creates a brand-new modpack on the victim's system and returns a valid GUID that's required later on for game launch.

```
{
  "args": [{
    "GameId": 432,
    "Name": "PWNED",
    "Author": "Elliott <3",
    "GameVersion": "1.21.8",
    "ModloaderVersionString": "forge-58.0.1",
    "ProfileImagePath": null,
    "InstallSource": 0,
    "ModsToInstall": [],
    "GroupId": null
  }],
  "type": "method",
  "name": "createModpack"
}
```

Step 2: Launching with attacker-controlled JVM arguments

Once that GUID comes back, it gets fed directly into a `minecraftTaskLaunchInstance` call. This is where things get fun, since the launcher happily accepts arbitrary JVM arguments via `AdditionalJavaArguments`.

For the demo, I used the following JVM flags to achieve code execution:

```
-XX:MaxMetaspaceSize=16m
-XX:OnOutOfMemoryError="cmd.exe /c calc"
```

By forcing the metaspace to an unrealistically small size with the first flag, the JVM quickly runs out of memory during startup. The second flag handles that memory error and executes an arbitrary command when the JVM crashes. In this case, it simply opens the classic `calc.exe`.

The malicious JVM arguments and GUID are then passed into the following launch payload:

```
{
  "args": [{
    "MinecraftInstanceGuid": "GUID",
    "ResolutionWidth": 1024,
    "ResolutionHeight": 768,
    "LauncherVisibility": "Close",
    "LauncherType": "Classic",
    "AdditionalJavaArguments": "-XX:MaxMetaspaceSize=16m -XX:OnOutOfMemoryError=\"cmd.exe /c calc\""
  }],
  "type": "method",
  "name": "minecraftTaskLaunchInstance"
}
```

When this message is sent, CurseForge launches the newly created modpack with the supplied JVM arguments, causing the payload to execute on the client system as the game starts. Depending on the user's launcher settings, this may require minimal user interaction, but in many cases it occurs automatically.

The included video PoC makes every step visible for demonstration. In a real-world attack, the same exploit could be executed entirely in the background by a malicious website.

Port Discovery

The only real wrinkle in this proof of concept is that CurseAgent doesn't bind its WebSocket server to a fixed port. Instead, it listens on a randomly assigned local port each time the launcher starts.

To address that, the PoC first needs to determine which port the WebSocket server is actually listening on. This is done by probing the local port range and attempting to open WebSocket connections until a valid CurseForge endpoint responds. Once the correct port is found, the rest of the exploit works normally.

The scan touches roughly 16,000 ports and works reliably in Chromium-based browsers, which handle the connection attempts without many issues, aside from some lag on lower-end systems. Firefox, on the other hand, crashes almost immediately, making the PoC largely unusable there (unless you somehow know the port).

Disclosure Timeline

Needless to say, this issue was reported to CurseForge shortly after discovery.

- **July 29, 2025** - Vulnerability discovered and initially reported.
- **August 11, 2025** - Follow-up communication with CurseForge. Due to the absence of a security contact, coordination was handled through an amazing community manager who escalated it internally.

- **August 31, 2025** - Vulnerability formally acknowledged.
- **September 15, 2025** - Received a CurseForge t-shirt.
- **November 2, 2025** - Fix released in the CurseForge app version [1.289.3](#), with origin checking added after follow-ups.
- **December 23, 2025** - Public disclosure after allowing time for rollout.

From initial report to fix, this left an exploitable RCE reachable from the browser for a little over three months.

Final Notes

To their credit, the CurseForge team ultimately addressed the issue, and the launcher no longer exposes this WebSocket server. The disclosure timeline was longer than I would have liked, given the impact, but I appreciate that it was eventually resolved.

Also, shoutout to CurseForge for the t-shirt!

- [#security](#)
- [#rce](#)
- [#research](#)

[Hacker News X](#)

Archived from <https://elliott.diy/blog/curseforge/>. Rendered offline from the local Markdown copy.